>_ CYBER TECH ACADEMY

Ethical Hacking Foundations

A responsible methodology for authorised security assessments, evidence gathering, validation, and reporting.

LEVEL: INTERMEDIATE

Creator: Dhanush | **Edition:** 1.0 | **Cost:** Free

How to use this guide

Read the concepts first, run examples only in a system you own or are authorised to use, complete the lab, then answer the review questions without looking back. Commands and outputs can differ by Linux distribution, software version, and local configuration.

Responsible learning: only test systems for which you have explicit permission. These notes are educational and do not grant permission to access, scan, disrupt, or modify third-party systems.

Learning objectives

- Explain authorisation, scope, rules of engagement, and test boundaries.
- Plan a structured assessment from discovery to reporting.
- Distinguish asset discovery, service identification, and vulnerability validation.
- Understand common web risks and secure testing principles.
- Write actionable findings with evidence, impact, and remediation.

Contents

- 1. Ethics, permission, and scope
- 2. The assessment lifecycle
- 3. Asset and service discovery concepts
- 4. Vulnerability assessment and validation
- 5. Web security concepts
- 6. Reporting and remediation
- Guided practical lab
- Knowledge check
- Glossary

1. Ethics, permission, and scope

Ethical hacking is security testing performed with explicit permission, a defined purpose, and agreed limits. A tool being publicly available does not make its use against a target lawful or authorised. Permission should identify the systems, domains, IP ranges, accounts, time windows, allowed techniques, prohibited actions, emergency contacts, and data-handling requirements.

Rules of engagement define how testing will be conducted. They should address rate limits, production-system safety, test accounts, data access, evidence storage, third-party services, reporting channels, and stop conditions. If scope is unclear, stop and obtain written clarification.

Never test a third-party website, Wi-Fi network, account, or public IP range without explicit authorisation. This textbook uses fictional examples and local or purpose-built lab targets.

2. The assessment lifecycle

A practical assessment typically includes planning and scoping, reconnaissance, asset and service discovery, vulnerability analysis, controlled validation, impact assessment, reporting, remediation support, and retesting. Not every engagement uses every phase, and the scope determines what is appropriate.

Reconnaissance gathers information about assets and exposure. Passive methods use information available without directly interacting with target systems; active methods interact with systems and therefore need explicit scope and care. Discovery results are hypotheses that require verification, not automatic proof of a weakness.

Keep a testing journal: timestamp, tester, target within scope, action, observed result, evidence reference, and any deviation. Avoid storing unnecessary personal data. Protect screenshots, logs, and exported data as sensitive material.

3. Asset and service discovery concepts

Asset inventory answers: what systems exist, who owns them, why they are needed, and how important they are? Service identification adds information about listening applications, versions, protocols, and exposure. The purpose is to understand the authorised attack surface, not to collect data indiscriminately.

Version banners can be incomplete, changed, or misleading. A reported version is not sufficient proof that a vulnerability is present: vendors backport fixes, configurations differ, and a service may be protected by other controls. Validate with reliable evidence and avoid disruptive tests on production systems.

Prioritise assets by business importance, exposure, data sensitivity, and the consequences of compromise. An internet-facing authentication system may deserve more attention than a low-impact isolated test machine.

4. Vulnerability assessment and validation

A vulnerability is a weakness that could affect confidentiality, integrity, or availability. Assessment combines automated results with manual verification, configuration review, version evidence, and context. False positives and false negatives are normal possibilities; findings should communicate confidence and limitations.

Severity is not the same as business risk. A severity score can help compare technical properties, but risk also depends on exposure, exploitability, required privileges, affected data, business impact, and compensating controls.

Validation should use the least intrusive method that proves the issue. Prefer safe test accounts and non-destructive proof. Do not extract real customer data, modify production records, establish persistence, or cause denial of service unless a specific activity is explicitly authorised and safely controlled.

5. Web security concepts

Web applications combine browsers, servers, APIs, databases, identity systems, and third-party services. Common risk areas include broken access control, authentication failures, injection, cross-site scripting, insecure configuration, cryptographic mistakes, vulnerable components, and insufficient logging.

SQL injection can occur when untrusted input is concatenated into a SQL statement. The main defensive control is parameterised queries, with least-privilege database accounts and careful error handling as supporting controls. Input validation is valuable but should not replace parameterisation.

Cross-site scripting (XSS) occurs when untrusted data is interpreted as executable browser content. Context-aware output encoding, safe templating, careful DOM APIs, and a well-designed Content Security Policy can reduce risk. Avoid inserting untrusted strings as raw HTML.

Broken access control occurs when a server fails to enforce what a user is allowed to do. Hiding a button in the interface is not authorisation. Check access on the server for every protected resource and action.

6. Reporting and remediation

A useful finding includes a clear title, affected asset, severity and rationale, preconditions, concise reproduction steps within scope, evidence, business impact, remediation, and retest result. Avoid vague statements such as 'the system is insecure' without evidence.

Remediation should address root cause. Examples include parameterised SQL queries, server-side object-level authorisation, secure session settings, dependency updates, removal of unnecessary exposure, stronger authentication, and centralised monitoring.

A retest verifies whether the fix works and whether it introduced regressions. Mark findings as fixed only when evidence supports the claim. If a risk is accepted rather than fixed, record the decision owner and review date.

Guided practical lab

Build a report for this fictional, authorised lab finding: a demo web application lets a user view another test user's profile by changing an object identifier.

- Write a neutral finding title.
- State the test environment and the two test accounts used.
- Describe the observed access-control failure without accessing real users' data.
- Explain confidentiality impact and any relevant limitations.
- Recommend server-side object-level authorisation checks and negative tests.
- Define retest evidence that would demonstrate the fix.

A professional report is factual and reproducible. Do not exaggerate impact, publish sensitive evidence, or test beyond the agreed scope.

Knowledge check

Try to answer these before checking the answer key. Write your reasoning, not just a single word.

1. What should be agreed before an ethical hacking assessment begins?

Answer: _____

2. Why is a version banner not enough to prove a vulnerability?

Answer: _____

3. What is the primary defence against SQL injection?

Answer: _____

4. Why is hiding an admin button not an access-control fix?

Answer: _____

5. What makes a security finding useful?

Answer: _____

Answer key and reasoning

1. Written authorisation, scope, rules of engagement, allowed methods, timing, data handling, contacts, and stop conditions.

2. Banners can be inaccurate, fixes may be backported, and configuration or compensating controls affect actual exposure.

3. Use parameterised queries or prepared statements rather than concatenating untrusted input into SQL.

4. The server must enforce authorisation on every protected action and resource; client-side hiding can be bypassed.

5. Clear scope, reproducible evidence, impact, severity rationale, actionable remediation, and retest criteria.

Glossary

Term	Meaning
Scope	Systems and activities included in an assessment.
Rules of engagement	Agreed procedures and limits for testing.
False positive	A reported issue that is not actually present as described.
Least intrusive validation	The safest method that adequately verifies a finding.
Remediation	Action taken to correct or reduce a weakness.
Retest	Verification that remediation is effective.

Further study

Continue with the next Cyber Tech Academy module, practise in a disposable virtual machine or an explicitly authorised lab, and consult official manuals for the tools you use. Verify commands against your own system before running them.

Cyber Tech Academy • Free learning • Learn the theory. Practise safely. Build real skills.